

Methodology and Technical Design for Learning-Augmented Robust Traffic Engineering

Detailed Design, Algorithms, Optimization, and Implementation Plan

ECE GY 7363: Communications Networks II - Design and Algorithms

April 2026

Focus	Algorithms, design formulation, optimization methods, and implementation plan
Core Model	Dynamic multi-commodity flow routing on a capacitated graph
Main Methods	LSTM traffic prediction, LP routing optimization, resilience testing

Abstract

This document presents the detailed methodology for the proposed project on learning-augmented robust traffic engineering in software-defined networks. It explains the design formulation, optimization model, machine learning techniques, baseline algorithms, resilience evaluation strategy, implementation architecture, and experimental methodology. The project combines time-series traffic prediction with multi-commodity flow optimization to proactively route traffic in a capacitated network. The primary design goal is to minimize congestion while preserving feasibility and resilience under demand variations and link failures.

1. Project Goal

The project studies how to improve traffic engineering decisions in a communication network when traffic demand varies over time and the network may experience failures. The central idea is to combine prediction and optimization in an end-to-end control loop: historical traffic, traffic prediction, routing optimization, and resilience evaluation.

The key hypothesis is that a predictor trained on past traffic matrices can estimate near-future demand well enough to help the controller make better routing decisions than methods that react only to the current state.

2. System Model

- The network is represented as a directed graph $G = (V, E)$, where V is the set of routers or switches and E is the set of directed links.
- Each link (i, j) has capacity $c_{ij} > 0$, and the network operates over discrete time epochs.
- Each source-destination pair is treated as a commodity k with source s_k , destination t_k , and time-varying demand $d_k(t)$.
- The full set of demands at time t forms the traffic matrix, and the ML component predicts one-step-ahead demand values $\hat{d}_k(t+1)$.

3. Design Formulation

The routing design uses continuous flow variables $f_{ij}^k(t)$, where each variable denotes the amount of commodity k routed on link (i, j) at time t .

Flow conservation ensures that each source injects its traffic, each destination receives it, and each intermediate node preserves balance.

Capacity constraints ensure the total traffic on each link does not exceed the physical link capacity.

Nonnegativity constraints require all routed flows to be nonnegative.

The primary objective is to minimize the maximum link utilization through an auxiliary variable $U(t)$, leading to a standard min-max congestion traffic engineering model.

4. Optimization Techniques

The primary optimization method is linear programming because the routing variables are continuous and all constraints and objectives are linear.

The project uses learning-augmented optimization, where the ML predictor provides the next traffic matrix and the optimizer computes capacity-feasible routes based on that prediction.

The routing loop is implemented as rolling-horizon control: observe traffic, predict the next state, optimize routing, evaluate realized performance, and repeat.

Optional extensions include robust optimization for uncertainty and fairness-aware formulations for equitable flow allocation.

5. Algorithms and Techniques

- Shortest-path routing with Dijkstra's algorithm serves as the simplest routing baseline.
- A current-demand-only LP baseline isolates the value of optimization without prediction.
- A conservative non-predictive LP baseline provides a basic robust benchmark.
- The prediction layer compares moving average, linear regression, optional MLP, and LSTM as the main forecasting model.
- GRU may be considered as an optional lighter recurrent alternative.

6. Priority of Advanced ML Models

Priority 1: Transformer-based time-series model. This is the best modern extension because traffic matrices are multivariate time series with long-range temporal dependencies, periodicity, and bursts. It is highly relevant to communication networks and relatively practical to add after LSTM.

Priority 2: Graph Neural Network or Spatio-Temporal GNN. This is highly network-relevant because the system itself is a graph, so a GNN can use topology information directly to model how traffic correlations propagate across connected nodes and links.

Priority 3: Graph Transformer or Spatio-Temporal Graph Transformer. This is the most ambitious option because it combines temporal attention with graph structure learning, but it also has the highest implementation and tuning complexity.

Recommended hierarchy: implement LSTM first, add a Transformer as the main latest-model comparison, then consider a GNN for stronger network structure awareness, and treat the Graph Transformer as a stretch goal.

7. Failure Analysis and Resilience Evaluation

Resilience will be tested under random single-link failures, critical-link failures based on utilization or graph centrality, and combined traffic-spike plus failure stress cases.

These scenarios will show whether the learning-augmented optimizer remains effective when the network deviates from nominal conditions.

8. Performance Metrics

- Maximum link utilization
- Average link utilization
- Number of overloaded links
- Fraction of demand satisfied
- Average path length
- Jain's fairness index
- Performance degradation under failure
- Prediction error using MAE, RMSE, and relative error

9. End-to-End Algorithm

- Load the topology and capacities.
- Generate or load time-varying traffic matrices.
- Split the traffic sequence into training, validation, and test periods.
- Train the traffic prediction model.
- For each test step, predict the next traffic matrix, solve the routing LP, evaluate realized performance, and simulate failure scenarios.
- Repeat the process for each baseline and compare methods statistically and visually.

10. Implementation Architecture

The codebase will be organized into modules for topology handling, traffic generation, predictors, optimization, baselines, failure simulation, metrics, experiments, and plotting.

The software stack will use Python with networkx, numpy, pandas, PyTorch, cvxpy or pulp, matplotlib, and seaborn.

The data flow will go from traffic generation to prediction, from prediction to optimization, and from optimization to evaluation and plotting.

11. Experimental Design

Experiments will use standard backbone-style topologies such as NSFNET, Abilene, or GEANT, along with optional synthetic stress-test graphs.

Traffic traces will be generated using base demand, periodic modulation, random noise, bursts, and hotspot pairs to create realistic but reproducible dynamics.

The traffic sequence will be split into training, validation, and test periods, and failure studies will include random and critical-link scenarios.

12. Complexity and Practical Considerations

The LP grows with the number of commodities and links, so early experiments will use medium-size networks and may restrict the number of active commodities if necessary.

The LSTM adds training cost but relatively low inference cost, making it practical for repeated control once trained.

13. Expected Findings

- Optimization-based routing should outperform shortest-path routing under congestion.
- Prediction-enhanced routing should outperform current-demand-only routing when traffic contains temporal structure.
- Under severe prediction error or extreme failures, conservative baselines may remain competitive.
- Resilience analysis will reveal when learning-augmented routing improves or harms robustness.

14. Risks and Mitigation

Optimization may become slow on large topologies, so the project will begin with medium-size graphs and scale carefully.

Synthetic traffic may be too noisy for ML to help, so the generator will include controlled temporal structure and forecasting quality will be measured independently.

Full robust optimization may be too ambitious, so scenario-based resilience evaluation is prioritized first.

15. Why the Technical Plan is Strong

The methodology combines a rigorous mathematical design formulation, classical optimization, modern ML, systematic baseline comparisons, explicit resilience analysis, and a realistic implementation plan.

That makes the project academically strong and well aligned with the course expectations.

16. Conclusion

This methodology document provides the full technical blueprint for carrying the project from idea to implementation and evaluation. The design formulation is a dynamic multi-commodity flow model on a capacitated graph, the main optimization technique is linear programming with a min-max utilization objective, and the main ML technique is LSTM-based traffic prediction.

Planned Code Modules

Module	Responsibility
topology.py	Load topology, capacities, and graph metadata
traffic_generator.py	Generate synthetic time-varying traffic matrices
predictors.py	Implement moving average, regression, and LSTM models
optimizer.py	Solve LP-based routing problems
baselines.py	Implement shortest-path and non-predictive baselines
failure_simulator.py	Run link-failure scenarios and resilience tests

metrics.py	Compute congestion, fairness, and ML metrics
run_experiments.py	Coordinate the full experiment pipeline
plots.py	Produce comparison figures and summary visuals

Table 1. Proposed implementation modules for the project pipeline.